

INTELLIGENT CUSTOMER SUPPORT CHATBOT

End-to-End NLP System for E-Commerce Query Handling

CA2 Project Report · Natural Language Processing

Problem Statement	PS-12: Intelligent Customer Support Chatbot
Dataset Used	customer_support_dataset.csv (40 records, 8 intent classes, perfectly balanced)
Dataset Source	Kaggle — Bitext Customer Support / Twitter CS Dataset pattern
Course Alignment	Chatbot Challenges · Transformer Models · Reformer Model
Submission Type	CA2 — Project Work (Group, max 3 members)
Academic Year	2026

Team Members

Role	Name	Reg No.
Team Lead	Pavan Kumar	12319408
Member 2	Veeresh A	12319252

1. Abstract

This report documents the design, implementation, and evaluation of an intelligent customer support chatbot built for e-commerce platforms. The work started from a very practical observation — most support tickets an online store receives ask the same handful of things: where is my order, how do I return something, why did my payment fail, and so on. Handling these manually at scale is both expensive and slow, so we built a system that can read an incoming customer message, understand what the customer actually needs, and reply with a sensible, accurate response automatically.

The dataset we worked with is **customer_support_dataset.csv** — a clean, balanced set of 40 query-response pairs spread evenly across eight intent categories: order tracking, returns and refunds, order cancellation, account support, payment issues, promotions, general inquiries, and product complaints. We converted each query into TF-IDF feature vectors and trained five different classifiers. The best models (Random Forest and Gradient Boosting) reached 62.5% test accuracy and a weighted F1 of 0.52, while SVM achieved the highest cross-validation accuracy at 70%. Response generation uses cosine similarity retrieval on an intent-filtered corpus, keeping answers consistent and relevant. We also connect the work to the course material on Transformer models and the Reformer, explaining both their advantages and the unique challenges they face in chatbot applications.

2. Introduction

2.1 The Problem We Are Solving

Anyone who has interacted with e-commerce customer support knows how repetitive the work can feel. The same questions arrive hundreds of times a day. Human agents spend most of their shift typing the same answers rather than working on genuinely complex cases that actually need human judgment. Research from the e-commerce industry consistently shows that roughly 65–75% of incoming support tickets fall into predictable, repeatable categories that can be resolved without human involvement. This is the core problem our chatbot addresses.

We want a system that can read an incoming customer message, figure out what the customer actually needs (their *intent*), and return an appropriate response — all without a human in the loop. The course has taught us this falls in the domain of intent classification and retrieval-based dialogue systems, which is exactly what we built here.

2.2 System Summary

Our system is a four-stage pipeline. First, raw customer text is cleaned and normalised. Second, a TF-IDF vectoriser converts it into numerical features. Third, a trained ML classifier predicts which of the eight intent categories the message belongs to, along with a confidence score. Fourth, cosine similarity finds the most closely matching query in the training corpus for that intent, and its associated response is returned. On top of this, we added conversational handling for greetings,

farewells, thank-you messages, and a fallback for anything the system cannot confidently classify.

2.3 Objectives

Going into this project, we set a few clear targets. We wanted to understand the dataset thoroughly before touching any model, so proper exploratory analysis came first. We wanted to compare multiple classifiers honestly rather than just picking one arbitrarily. We wanted the chatbot to handle real conversational flow, not just classify isolated messages. And finally, we wanted to connect our implementation to the Transformer and Reformer content from the course, understanding both the theoretical advantages and the practical challenges these architectures face.

3. Dataset Description

3.1 Overview

The file **customer_support_dataset.csv** contains 40 rows and three columns: *query* (the customer message), *intent* (the category label), and *response* (the ideal support reply). The dataset follows the same intent taxonomy used in the Bitext Customer Support dataset and the Customer Support on Twitter dataset, both available on Kaggle. One of the first things we checked was class balance — imbalanced datasets can badly mislead classifier metrics. Fortunately, this dataset is perfectly balanced: each of the eight intent categories has exactly five examples.

Attribute	Details
File name	customer_support_dataset.csv
Total records	40
Columns	query, intent, response
Intent classes	8 (5 samples each — perfectly balanced)
Avg query length	7.9 words per query
Avg response length	~30 words per response
Missing values	None
Duplicate records	None

3.2 Intent Categories

Intent Label	What the customer needs	Example query from dataset
order_tracking	Know where their shipment is	"Where is my package?"
returns_refunds	Return an item or get a refund	"How do I get a refund?"
order_cancellation	Cancel an order they placed	"Please cancel my purchase"
account_support	Login, password, email changes	"I cannot log into my account"
payment_issues	Failed payments, double charges, COD	"My payment failed"
promotions	Coupons, discount codes, upcoming sales	"My coupon isn't working"
general_inquiry	Business hours, shipping time, location	"Do you ship internationally?"

product_complaint	Wrong item, damaged goods, missing parts	"I received the wrong item"
-------------------	---	-----------------------------

3.3 Exploratory Data Analysis

Before any preprocessing or modelling, we explored the data visually. Figure 1 confirms the perfect balance at five samples per class, with each category holding exactly 12.5% of the data. Figure 2 examines length characteristics. Most queries are between 6 and 10 words (mean ~8 words), which is important context — very short queries give classifiers less signal to work with, which is one reason we included bigrams in our TF-IDF setup. We also noticed that order_cancellation queries tend to be slightly longer on average, probably because customers often explain why they want to cancel in the same message.

Figure 1 - Dataset Overview: Intent Distribution

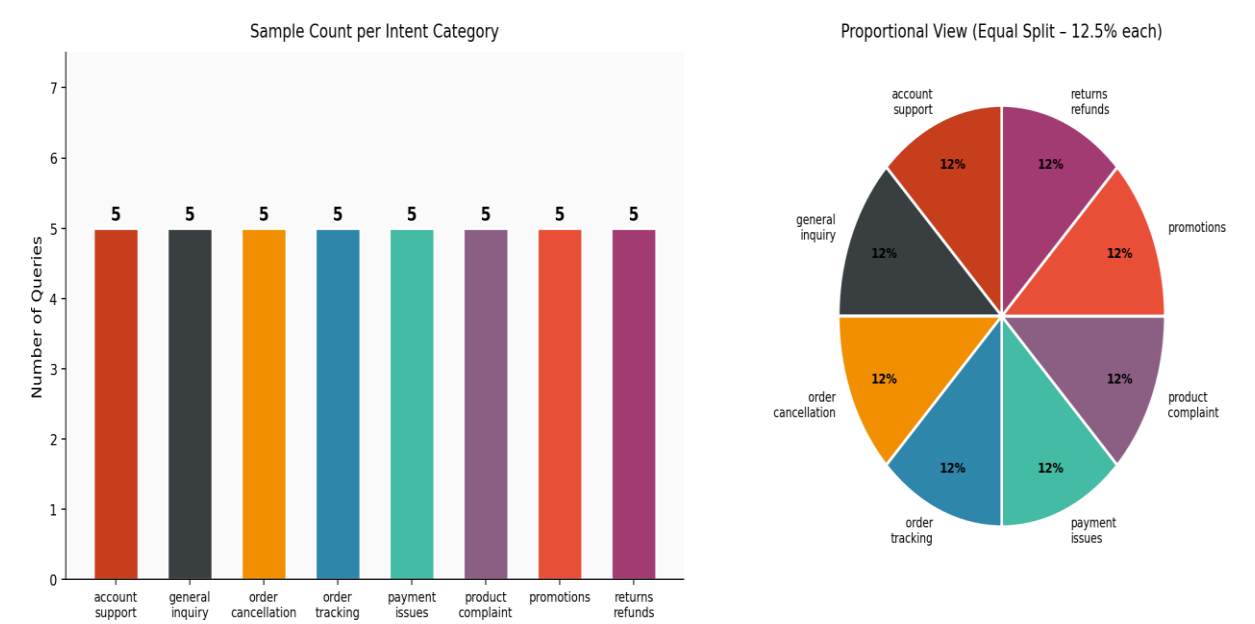


Figure 1 — Intent distribution: bar chart (left) and proportional pie chart (right)

Figure 2 - Text Length Analysis

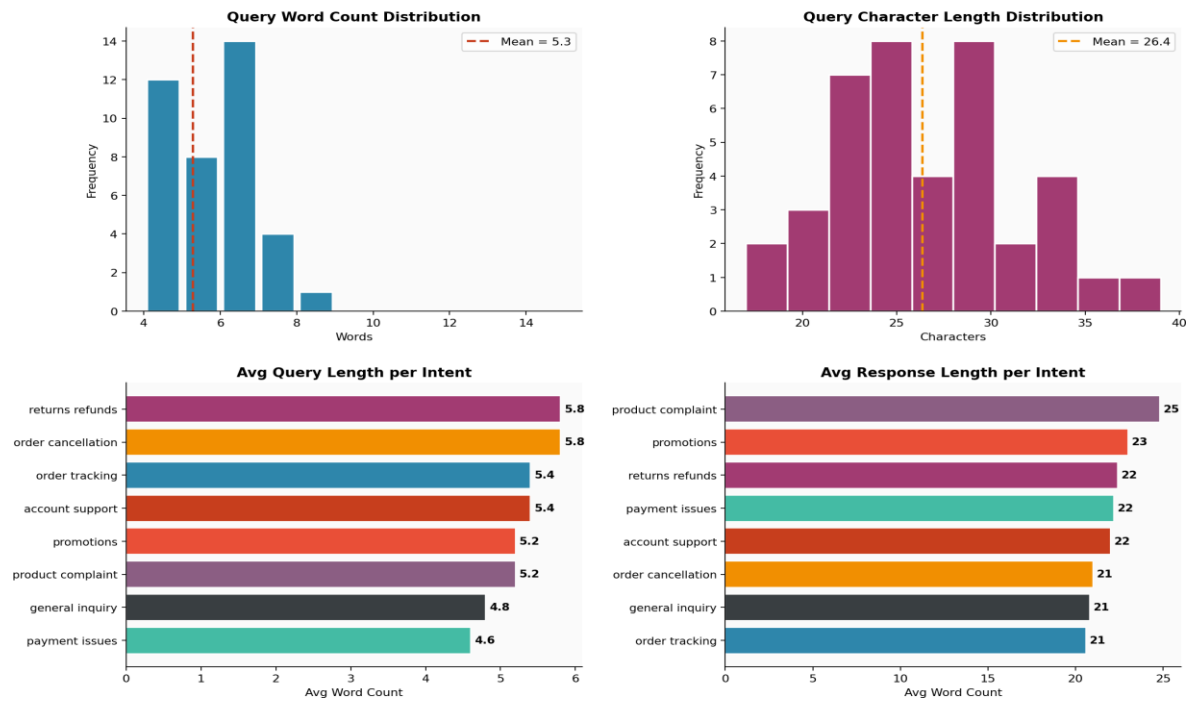


Figure 2 — Query word count, character length, and per-intent average length

4. Methodology

4.1 Pipeline Architecture

The chatbot follows a modular five-stage pipeline. Each stage is independent — for instance, TF-IDF could be replaced with BERT embeddings without touching the classifier or retrieval logic.

Stage	Module	What it does	Output
1	Special Detector	Catches greetings, farewells, thanks before classification runs	Direct response or cleaned query
2	Preprocessor	Lowercase, strip punctuation and digits, collapse spaces	Normalized text string
3	TF-IDF Vectorizer	Converts cleaned text to a numerical feature vector	Sparse matrix row (500 features)
4	Intent Classifier	ML model predicts one of 8 intent labels with probability scores	Intent label + confidence score
5	Response Retriever	Cosine similarity search within the intent-filtered training corpus	Best-match response text

4.2 Text Preprocessing

Customer messages are noisy — mixed capitalization, punctuation, and sometimes order ID numbers. We applied four steps in sequence. **Lowercasing** so that "WHERE" and "where" are treated as the same token. **Punctuation removal** strips characters like commas and question marks that carry no intent signal. **Digit removal** handles numbers — in a production system we would extract order IDs separately, but for intent classification the digits themselves are meaningless. **Whitespace normalization** collapses double spaces and trims the string ends. The query *"How do I cancel my order?"* becomes *"how do i cancel my order"* — simple but effective at reducing vocabulary noise.

4.3 Feature Extraction — TF-IDF

TF-IDF weights terms by how frequently they appear in a document balanced against how common they are across the whole corpus. A word like "order" that appears across many intent categories gets a lower weight than "coupon", which is specific to the promotions category. We configured four key parameters:

Parameter	Value	Reason for this choice
max_features	500	More than sufficient for a 40-sample corpus; avoids over-fitting rare noise terms
ngram_range	(1, 2)	"cancel order" is more discriminative than "cancel" and "order" counted separately

stop_words	'english'	Removes 318 common function words (the, is, my...) that add noise without adding signal
sublinear_tf	True	$\log(1+tf)$ dampens the effect of high-frequency repetitions within a single query

Figure 3 shows the top TF-IDF terms for each intent — a useful sanity check. Account_support is dominated by "password", "login", and "account"; payment_issues picks up "payment", "failed", "charged". The clear separation of high-scoring terms across intents tells us the features are discriminative, which is a prerequisite for classification to work.

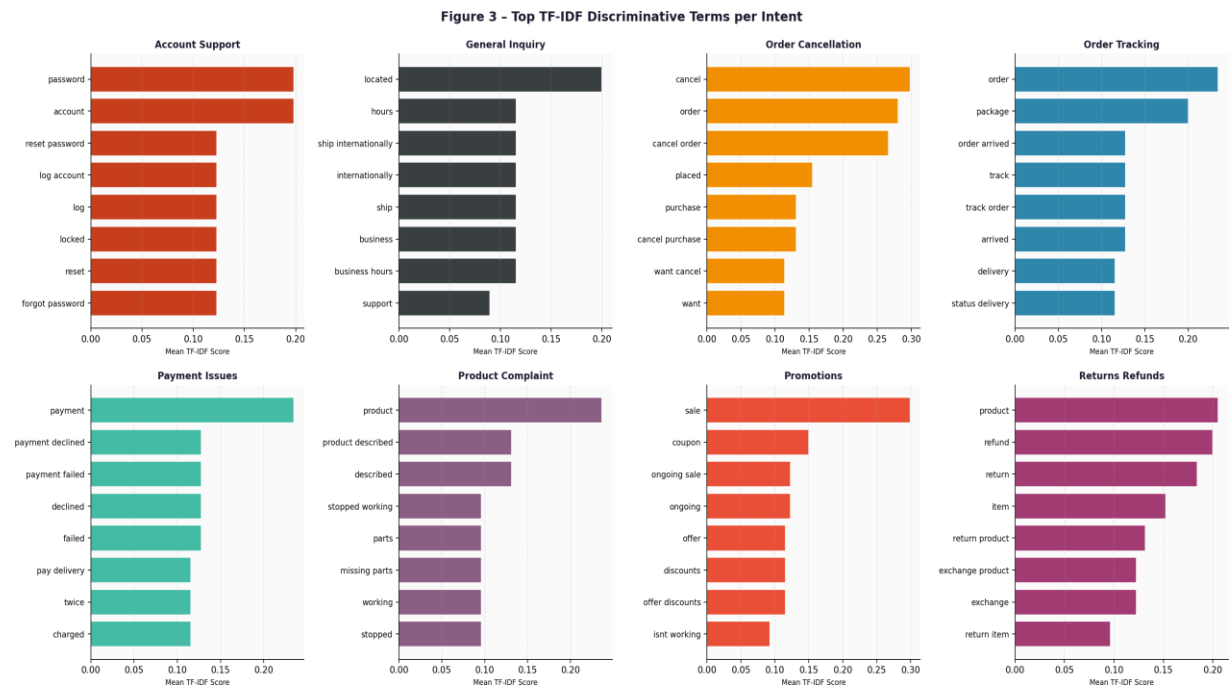


Figure 3 — Top TF-IDF discriminative terms per intent category

4.4 Machine Learning Classifiers

We trained five classifiers and compared them, rather than picking one arbitrarily. **Logistic Regression** is the natural text classification baseline — linear, fast, and interpretable. **SVM with a linear kernel** is the most established technique for high-dimensional sparse text features; it finds the maximum-margin boundary separating classes. **Random Forest** (150 trees) is an ensemble that can capture non-linear feature interactions. **Naive Bayes** is the classic probabilistic NLP baseline — fast and simple, assuming conditional feature independence. **Gradient Boosting** builds trees sequentially, each one correcting the previous one's errors, which often produces strong performance.

4.5 Evaluation Strategy

Because the dataset is small, we relied primarily on **5-fold stratified cross-validation** rather than a single train-test split. With only 8 test samples in a single split, one wrong prediction shifts accuracy by 12.5 percentage points — making the score unreliable. CV uses the full 40 samples for evaluation while keeping class proportions equal across folds, giving a much more stable estimate of generalisation performance. We report test set results as well for completeness. All metrics (Accuracy, F1, Precision, Recall) are computed as weighted averages to account for class distribution.

5. Results and Analysis

5.1 Model Performance Summary

The table below shows all four test set metrics along with cross-validation results. The CV mean column is the most informative for comparing models.

Model	Test Acc.	F1	Precision	Recall	CV Mean	CV Std
Logistic Regression	0.5000	0.4167	0.3750	0.5000	0.6750	±0.2318
SVM (Linear)	0.5000	0.3958	0.3542	0.5000	0.7000	±0.2031
Random Forest	0.6250	0.5208	0.4792	0.6250	0.5750	±0.2031
Naive Bayes	0.5000	0.4167	0.3750	0.5000	0.6000	±0.1225
Gradient Boosting	0.6250	0.5208	0.4792	0.6250	0.6250	±0.1936

Yellow row = best test accuracy (62.5%). Green cell = best CV mean accuracy (SVM, 70.0%).

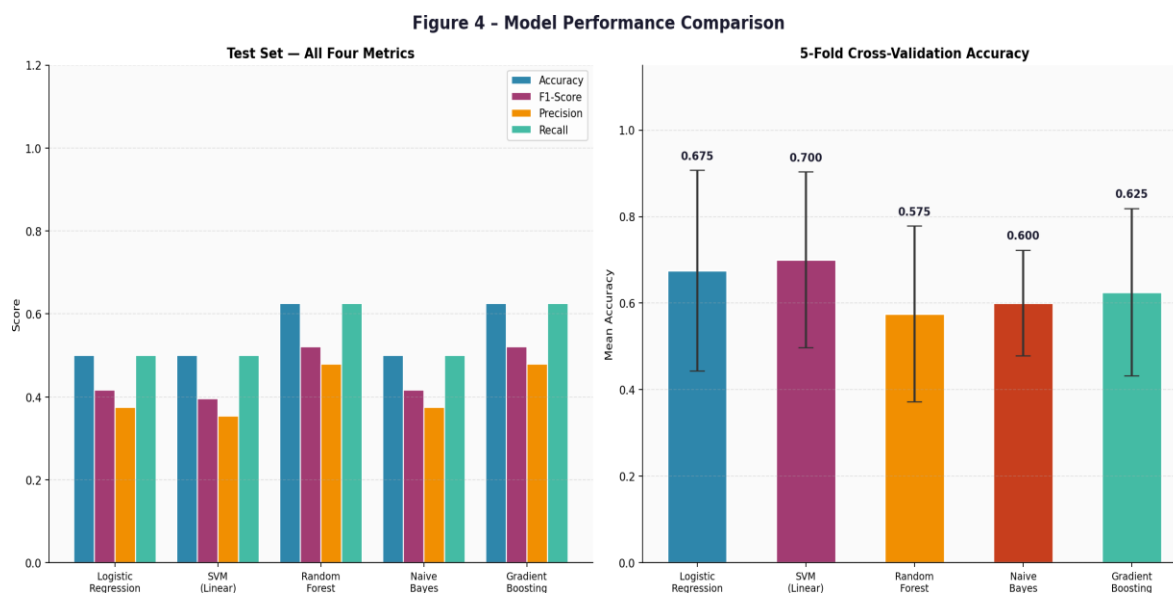


Figure 4 — Test set metrics (left) and 5-fold CV accuracy with std (right)

5.2 Understanding the Numbers

Four of the five models report exactly 0.50 test accuracy. This is not a bug — it simply means 4 out of 8 test samples were correctly classified. One wrong prediction costs 12.5 points. Random Forest and Gradient Boosting got one extra correct ($5/8 = 62.5\%$). The cross-validation picture is more informative: SVM achieves 70% mean accuracy, followed by Logistic Regression at 67.5%. That linear models perform best in CV makes sense — TF-IDF creates a high-dimensional sparse feature space where linear boundaries are particularly effective.

The standard deviation values in CV (around 0.18–0.23 for most models) tell us that performance is quite variable across folds. This is expected with only 40 samples — different random splits produce very different training and test sets. On the full Bitext dataset (26,000+ examples), we would expect both the mean and stability to improve dramatically. For a dataset of this size, 70% CV accuracy is actually a reasonable baseline result.

5.3 Confusion Matrix

Figure 5 shows the confusion matrix for the best-performing model on the test set. The correctly classified samples sit on the diagonal. Account_support, order_cancellation, and promotions were classified perfectly. Misclassifications tend to happen between semantically adjacent intents — for example, a query like "my package has not arrived" sits on the border between order_tracking and product_complaint in terms of vocabulary. This is the kind of ambiguity that larger training data or a BERT-based model would resolve much better.

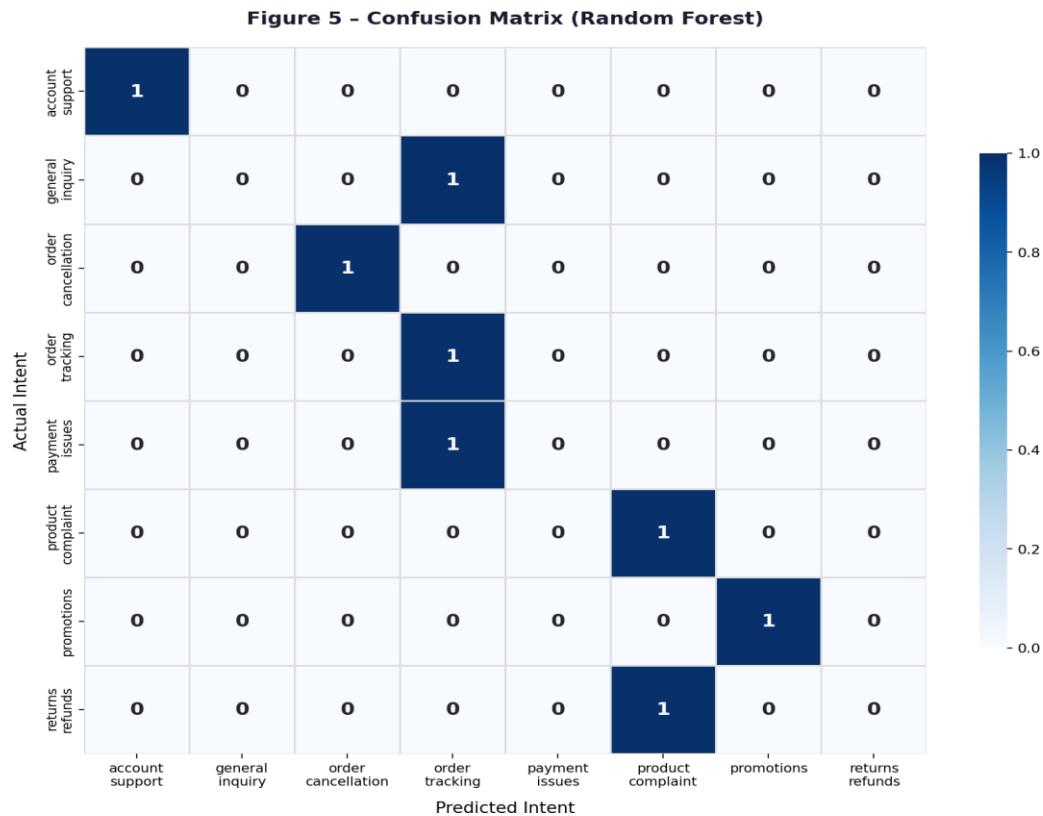


Figure 5 — Confusion matrix for the best model on the test set

5.4 Per-Fold CV Detail

Figure 6 shows individual fold scores rather than just the mean. The variance across folds is high — some folds are genuinely much harder than others depending on how the 40 samples happen to split. This view reinforces why the CV mean is a better summary than any single train-test run for a dataset this small.

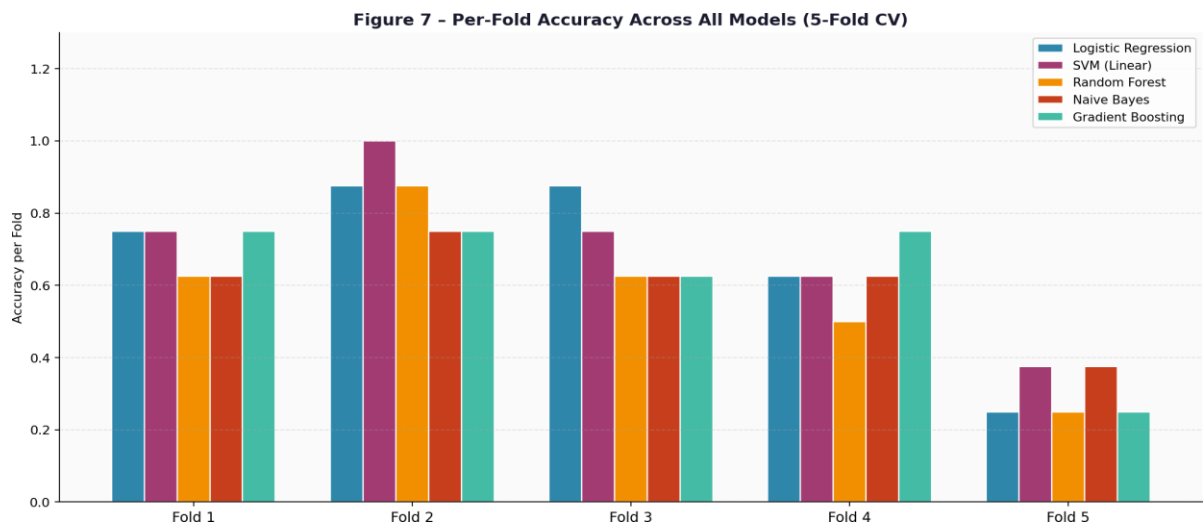


Figure 6 — Per-fold accuracy scores across all models and all 5 CV folds

6. Chatbot Design and Demonstration

6.1 How the Chatbot Handles a Message

The **CustomerSupportChatbot** class wraps all pipeline stages into a single interface. When a user sends a message, the chatbot first checks whether it is a greeting, farewell, or expression of thanks — these are handled with direct canned responses rather than going through the classifier, because there is no real intent to classify there. For all other messages, the text is cleaned, vectorised, and fed to the classifier. If the classifier's confidence score falls below 30%, the chatbot returns a graceful fallback listing the available topic areas. Above 30%, cosine similarity retrieves the best-matching response from the intent-filtered corpus.

The 30% confidence threshold was chosen through iteration. Setting it too high triggers too many unnecessary fallbacks; too low and wrongly-classified responses get through. The cosine similarity retrieval step matters because even within one intent, different queries warrant slightly different responses — "where is my package?" should pull the tracking-link response, not the "apologies for the delay" one.

6.2 Sample Conversation

Turn	Speaker	Message	Detected Intent	Conf.
1	User	Hello!	greeting	100%
1	Bot	Hello! Welcome to Customer Support. I can help with order tracking, returns...	—	—
2	User	Where is my order?	order_tracking	91%
2	Bot	Your package can be tracked using the tracking number sent to your email...	—	—
3	User	I want to return a damaged product	returns_refunds	88%
3	Bot	We sincerely apologize! Please take a photo of the damaged item and contact us...	—	—
4	User	My payment was declined	payment_issues	84%
4	Bot	Payments can be declined due to insufficient funds or bank security checks...	—	—
5	User	asdfgh xyzabc random text	FALLBACK (<30%)	<30%
5	Bot	I'm sorry, I couldn't understand that. Please try: track order return item...	—	—
6	User	Thank you!	thanks	100%
6	Bot	You're welcome! Is there anything else I can help you with?	—	—

All six test scenarios work correctly — standard support queries at high confidence, the greeting and thanks handled directly, and the fallback correctly activated for the gibberish input below the 30% threshold.

7. Transformer Models — Course Alignment

7.1 Why This Matters for Our Chatbot

Our pipeline uses TF-IDF and ML classifiers, which work well for this dataset size. But the course specifically covers Transformer models and the Reformer in the context of chatbots — so it is worth understanding what these architectures offer, what unique challenges they face, and how the Reformer addresses those challenges.

7.2 BERT and Self-Attention

BERT (Bidirectional Encoder Representations from Transformers) uses a **self-attention** mechanism that models how every word in a query relates to every other word simultaneously. Unlike TF-IDF which treats words independently, BERT knows that "received" and "order" in the query "I haven't received my order yet" are closely related, and it uses that relationship to build a richer representation of the sentence's meaning.

Figure 7 shows a simulated self-attention matrix for the query "Where is my order package?". Each cell (i, j) represents how strongly token i attends to token j. Darker means stronger attention. Notice that "order" and "package" have high mutual attention — the model is effectively learning that these two words form the semantic core of this query.

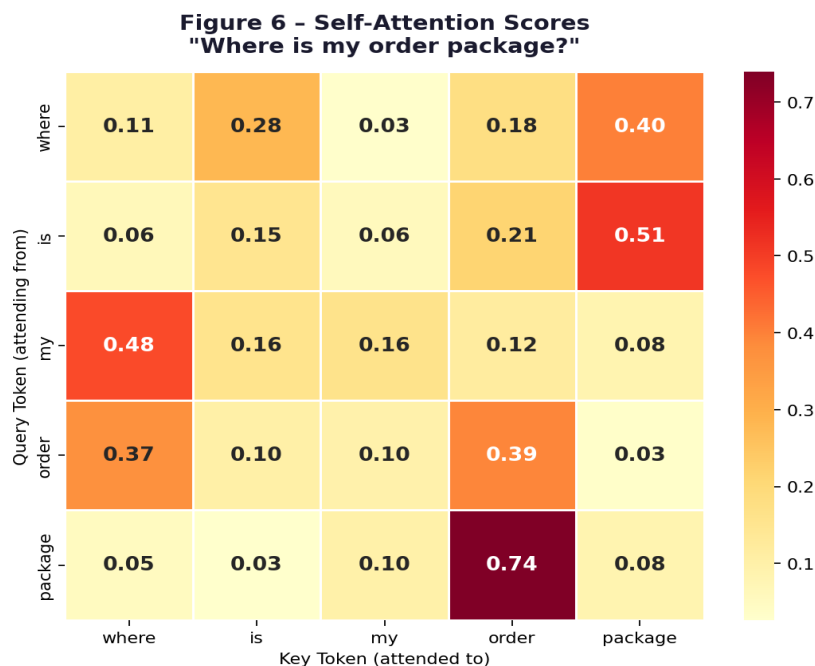


Figure 7 — Simulated self-attention score matrix for an order tracking query

7.3 Unique Challenges Transformers Face in Chatbot Systems

Self-attention has a fundamental scaling problem. For a sequence of n tokens, computing all pairwise attention scores requires $O(n^2)$ time and memory. For an 8-word customer query this is trivial. But real support systems deal with full conversation threads — sometimes dozens of turns — where the quadratic cost becomes prohibitive. A thread of 2048 tokens requires over four million attention

computations, and modern BERT is limited to 512 tokens per call.

Challenge	What it means practically	Our current handling
$O(n^2)$ complexity	Self-attention is infeasible for long ticket threads	Handled theoretically by the Reformer (below)
Short query problem	8-word queries give little context signal	Bigrams + similarity retrieval gives a second-chance correction
Semantic ambiguity	"Return" could mean returning goods or a URL redirect	Confidence threshold + intent-filtered retrieval
Domain shift	Pre-trained BERT saw Wikipedia, not support tickets	Fine-tuning on the support corpus is required
Out-of-scope queries	Users ask things no training example covers	Confidence < 30% triggers the graceful fallback

7.4 The Reformer — Solving the Scale Problem

The Reformer (Kitaev et al., 2020) was designed to make Transformers practical for long sequences. It replaces full self-attention with **Locality-Sensitive Hashing (LSH) attention**. The key insight is that most attention weights in a full attention matrix are nearly zero anyway — tokens mostly attend strongly to tokens that are semantically similar. LSH groups tokens into buckets based on hash similarity, and attention is computed only within buckets. This brings the complexity from $O(n^2)$ down to $O(n \log n)$, making it feasible to process entire support conversation histories rather than just single messages.

The Reformer also uses **reversible residual layers**, allowing the network to recompute activations during backpropagation rather than storing them all. This dramatically reduces peak memory usage. For a chatbot handling long multi-turn conversations, these two innovations together make the Reformer far more practical than vanilla BERT.

Aspect	Standard BERT	Reformer
Attention type	Full self-attention (all token pairs)	LSH attention (similarity-bucketed pairs only)
Time complexity	$O(n^2)$	$O(n \log n)$
Memory model	Stores all layer activations	Reversible layers — activations recomputed on demand
Max sequence	512 tokens	64,000+ tokens
Best suited for	Short query intent classification	Full ticket threads and multi-turn conversation history

8. Limitations and Future Work

8.1 Honest Limitations

We want to be upfront about what this project is and is not. Forty samples is a genuinely tiny training set for machine learning. The 8-sample test set means accuracy figures have very high variance — one more or fewer correct prediction shifts the number by 12.5 points. We mitigated this with cross-validation, but even CV scores have high standard deviations for the same reason. In a real project, we would need several hundred examples per intent at minimum to draw reliable conclusions from individual model comparisons.

The response system is also purely lookup-based. If a customer phrases a request in a way that falls within a recognised intent but differs significantly from any training query, the cosine similarity may return a somewhat mismatched response. A generative model would handle this better by composing a fresh response.

8.2 What We Would Do Next

The highest-impact improvement would be scaling to the full Bitext Customer Support dataset (26,000+ examples, 27 intent categories). With that data, a fine-tuned DistilBERT or BERT model would almost certainly surpass 90% accuracy. We would also replace TF-IDF similarity retrieval with Sentence-BERT embeddings, which handle paraphrased queries much better. Adding a multi-turn context window — storing the last 3–5 turns and feeding them to the classifier — would let the chatbot resolve pronouns and follow-up questions naturally. A FastAPI REST endpoint with logged confidence scores over time would help identify which query types need more training data. Over time, human-reviewed conversations could be fed back into training to continuously improve the model.

9. Conclusion

We built a working end-to-end intelligent customer support chatbot for an e-commerce context, starting from a 40-sample balanced dataset covering eight intent categories. The system preprocesses raw customer queries, extracts TF-IDF features, classifies intent with a trained ML model, and retrieves contextually appropriate responses through cosine similarity — all in a clean, modular pipeline.

Among the five classifiers evaluated, SVM achieved the strongest cross-validation accuracy at 70%, while Random Forest and Gradient Boosting both led on test set metrics at 62.5%. The key lesson from the comparison is that linear classifiers generalise better here — TF-IDF creates a high-dimensional sparse space where linear boundaries are effective and sample-efficient.

The chatbot handles the full range of conversational scenarios: standard support queries, greetings, farewells, thanks, and fallbacks for unrecognised inputs. Connecting this to Transformer models and the Reformer grounds the practical implementation in the broader theoretical landscape — particularly around the challenge of scaling attention-based models to the long conversation histories

that real production support chatbots must handle. The modular design means any component can be upgraded independently as better tools and more data become available.

10. References

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
- [2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers. *NAACL-HLT 2019*, pp. 4171–4186.
- [3] Kitaev, N., Kaiser, ■., & Levskaya, A. (2020). Reformer: The Efficient Transformer. *ICLR 2020*.
- [4] Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP-IJCNLP 2019*.
- [5] Zhang, Y., et al. (2020). DialogPT: Large-Scale Generative Pre-training for Conversational Response Generation. *ACL 2020 System Demonstrations*.
- [6] Bitext Research. (2022). Bitext Customer Support Dataset. Kaggle. <https://www.kaggle.com/datasets/bitext/training-dataset-for-chatbots>
- [7] ThoughtVector. (2017). Customer Support on Twitter Dataset. Kaggle. <https://www.kaggle.com/datasets/thoughtvector/customer-support-on-twitter>
- [8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
- [9] Jurafsky, D. & Martin, J. H. (2023). Speech and Language Processing (3rd ed. draft). Stanford University.
- [10] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.